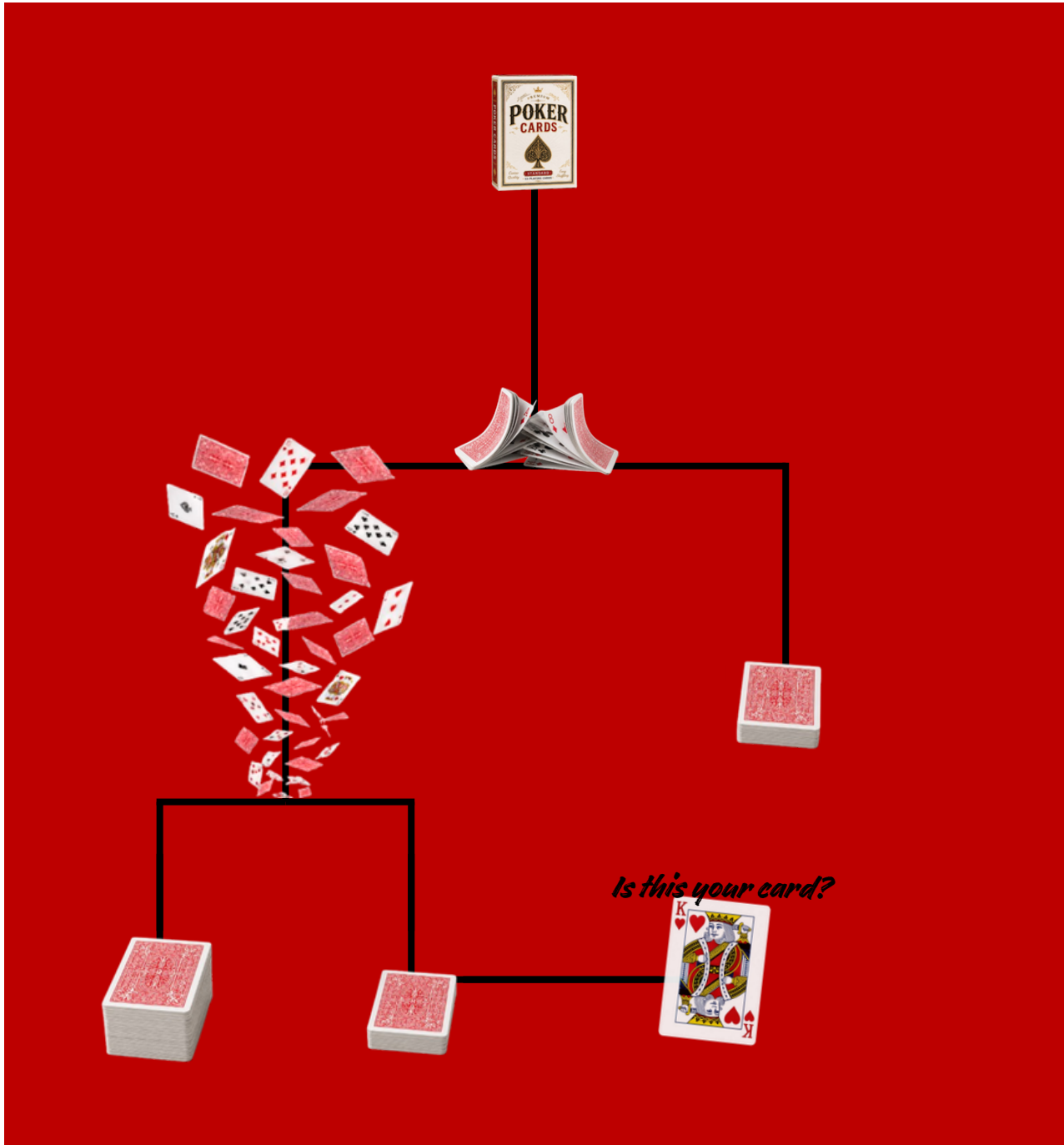


# DECODING THE DECK: CNN TRANSFER LEARNING WITH EFFICIENTNETB0 FOR POKER CARD RECOGNITION



Project Report By Paola María Garrido Montes

Course: Development of Advanced Applications of Computer Science

Professor: Benjamín Valdés Aguirre

Department of Computer Science Tecnológico de Monterrey, Campus Queretaro June 2026

## I. INTRODUCTION

Poker card classification presents several challenges because cards share similar visual structures, repeated suit symbols and fine details that can be difficult to distinguish. Previous studies have shown that model performance is strongly affected not only by dataset quality and distribution, but also by training time, computational resources, and generalization to unseen images (Castillo et al., n.d.; Mittal et al., 2024).

This project aims to address these challenges by developing an efficient deep learning model capable of classifying poker card images across 53 distinct classes. Instead of training a convolutional network from scratch, the project uses EfficientNetB0 with transfer learning and fine-tuning to take advantage of pre-trained ImageNet features while reducing training complexity. The dataset was balanced to provide equal representation across all classes, and the validation and test partitions were expanded to support a reliable evaluation process. By combining dataset balancing, targeted augmentation, regularization, and fine-tuning, the project seeks to achieve strong classification performance using a relatively small dataset and limited computational resources.

## II. DATASET

The selected dataset, **Cards Image Dataset-Classification**, is a collection of playing card images structured into **53 distinct classes**. These correspond to the combinations of **13 card ranks** (ranging from Ace to King) across **4 suits** (Clubs, Diamonds, Hearts, and Spades), alongside an additional class designated for the Joker. The dataset comprises a total of 8,154 JPG images, with each image having native dimensions of  $224 \times 224 \times 3$ . The data setup is partitioned into 7,624 training images, 265 validation images, and 265 test images.

## III. Dataset Balancing and Partitioning Strategy

The original distribution of the dataset showed noticeable class imbalance, with the number of sample images varying significantly across the 53 target classes. In deep learning architectures, training on unbalanced data can cause the network to optimize its weights preferentially toward the overrepresented categories. This may result in poor generalization and diminished performance on underrepresented classes. To reduce this risk and support fair class representation, dataset balancing was performed.

The number of images per class was analyzed across the original dataset splits. Since the analysis revealed that every class contained a baseline minimum of at least 118 images, this value was established as the threshold for each class. Although this down-sampling approach required discarding a portion of the images from larger classes, enforcing a balanced distribution helps ensure that the optimization algorithm weights every playing card class with equal importance, preventing the model from favoring high-frequency classes.

In addition, the original dataset partitioning was not ideal. The validation and test sets contained only 265 images each, which corresponds to 5 images per class. This limited number of samples may not provide a reliable estimate of the model's generalization performance, especially in a 53-class classification problem. A very small validation set can make model tuning unstable, while a small test set may produce evaluation results that are highly sensitive to individual image predictions.

To address these limitations, all source images were consolidated into class-specific root directories, and a custom pipeline was executed through the following sequential operations:

- **Information Retrieval:** Each class directory was scanned to count valid image formats.
- **Shuffling & Reproducibility:** The image paths within each class were randomly shuffled using a fixed random seed (seed=42), ensuring that the same data split could be generated across separate runtimes.
- **Sub-sampling & Splitting:** For each class, exactly 118 images were isolated and partitioned into three mutually exclusive subsets:
  - **Training Set:** 75 images per class (~64% of the sub-sample) to feed the neural network and update the model weights.
  - **Validation Set:** 19 images per class (~16% of the sub-sample) to monitor loss and accuracy metrics during training.
  - **Testing Set:** 24 images per class (~20% of the sub-sample) to serve as an unbiased final performance benchmark.

As a result, the final balanced dataset contained **6,254 images**, partitioned into **3,975 training images**, **1,007 validation images**, and **1,272 testing images**.

#### IV. PRE-PROCESSING AND DATA AUGMENTATION

This implementation leveraged a transfer learning approach using EfficientNetB0 as the base model. Therefore, all visual data had to follow the same preprocessing scheme used by the network during its original ImageNet pretraining. To ensure architectural compatibility, the native EfficientNet *preprocess\_input* function was integrated directly into the input pipeline, transforming raw pixel values into the distribution expected by the model's convolutional layers.

During the initial implementation phase, the network showed signs of overfitting, with a gap above 20 percentage points between training and validation performance. To mitigate this, data augmentation was introduced into the training stream to improve overall generalization capacity without overcomplicating the learning task, given that the original dataset already contained significant variations in rotation, zoom, brightness, contrast, texture, and sharpness. Additional augmentation operations were tested, but they did not improve training performance. Therefore, only the following targeted transformations were applied:

- **Zoom Range (0.08):** Simulates slight variations in camera distance.
- **Brightness Range [0.85, 1.15]:** Replicates changes in lighting, shadows, and reflections.

For the stream configuration, all inputs were set to a standard **224 × 224 resolution**, as required by EfficientNetB0. The data streams were grouped into **mini-batches of 32 images** to optimize memory usage and stabilize gradient calculations during backpropagation. Furthermore, the stream class mode was configured as *categorical*, allowing the folder directory structure to be translated into one-hot encoded labels compatible with the *Categorical Crossentropy* loss function. For the testing stream, image shuffling was disabled to preserve the exact sequential order of the test data and correctly map predictions to labels.



## V. MODEL

To maximize classification performance with a limited sample size of 75 training images per class, transfer learning was implemented using **EfficientNetB0** as the base model. EfficientNet was selected because the literature shows that its **compound scaling method** balances network depth, width, and resolution, rather than scaling only one dimension independently (Tan & Le, 2019). This allows the architecture to achieve a better balance between accuracy and computational efficiency, using fewer parameters than many traditional CNNs:

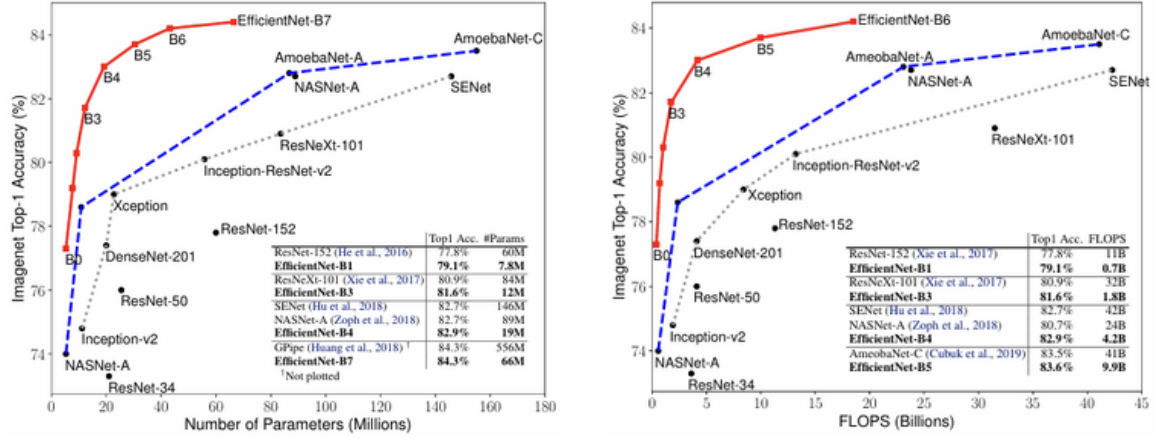


Figure 1. EfficientNet performance comparison based on model size and computational cost. (Tan & Le, 2019)

The **B0 variant** was implemented because it is the baseline and smallest EfficientNet model. Despite its size, it provides effective feature extraction through **MBConv blocks**, which help capture relevant visual patterns such as edges, textures, shapes, and fine-grained card details (Tan & Le, 2019). This makes it suitable for the dataset size and the computational limitations.

### Structure and Classification Head

The model architecture was constructed using the following sequential pipeline:

- **Feature Freezing:** The pre-trained EfficientNetB0 weights were frozen to preserve the generalized ImageNet features without modifying the pre-trained parameters.
- **Global Average Pooling:** This layer calculated the spatial average of the feature maps and compressed the output into a smaller vector, reducing the number of parameters and helping control overfitting.
- **Dense Layer:** The fully connected intermediate layer acted as the primary component for pattern translation, using 128 neurons and ReLU activation to learn patterns specific to the playing card dataset. In the final model, L2 regularization was applied with a value of 0.001 to penalize large weights and reduce overfitting.
- **Dropout Regularization:** This layer deactivated 50% of the neurons during each training pass to counteract overfitting, allowing the network to learn robust feature representations.
- **Output Classification:** The final output layer mapped the model outputs to the 53 target playing card classes, utilizing Softmax activation to generate the probability distribution across all classes.
- **Optimizer:** The Adam optimizer was used during compilation because it adjusts the learning process for each parameter, providing a good balance between stable convergence and efficient training.
- **Loss Function:** The model was compiled using Categorical Crossentropy, which is appropriate for multi-class classification problems with one-hot encoded labels.

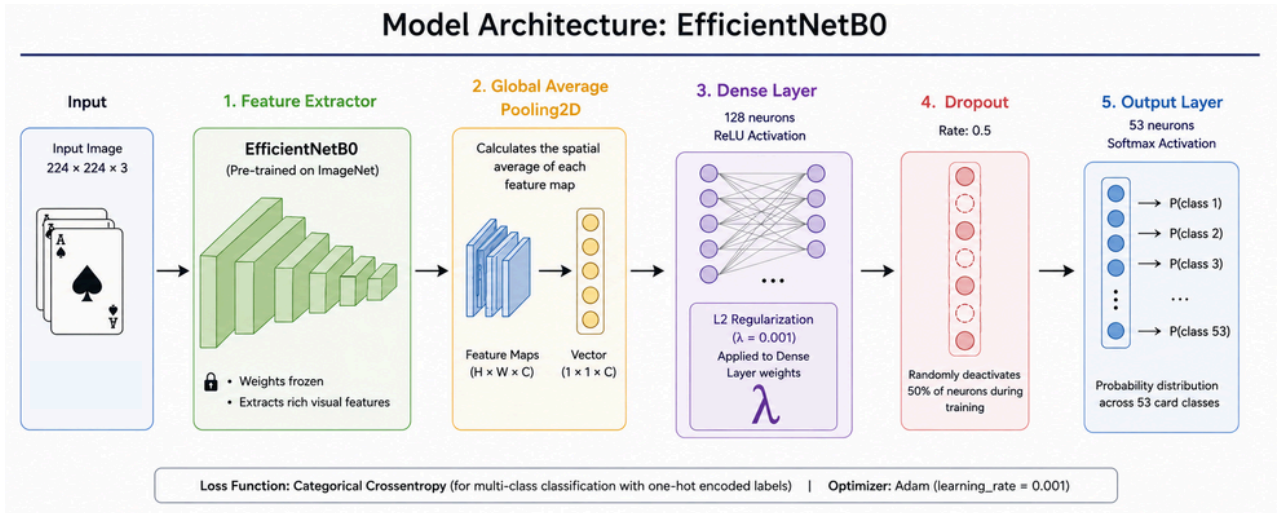


Figure 2. EfficientNetB0 architecture for poker card classification.

## VI. TRAINING

The network was trained for a maximum of 25 epochs using callbacks to monitor validation performance, preserve the best model parameters, and adaptively handle the learning rate.

- **Model Checkpoint:** Tracks validation loss and saves only the top-performing structural weights to Google Drive.
- **Early Stopping:** Monitors validation loss with a patience threshold of 5 epochs to prevent overfitting. If validation error fails to decrease for 5 consecutive epochs, it terminates execution and automatically restores the best weights recorded.
- **Learning Rate Decay:** Dynamically scales down the learning rate by a factor of 0.2 if the validation loss stagnates for 3 consecutive epochs.

Before applying L2 regularization, the first model achieved a **training accuracy of 77.23%** with a **loss of 0.7514** and a **validation accuracy of 60.18%** with a **loss of 1.5023**. After implementing L2 regularization, the final model achieved a **training accuracy of 73.23%** with a **loss of 1.2258** and a **validation accuracy of 59.98%** with a **loss: 1.8581**. Although the validation accuracy remained nearly unchanged and the loss increased, the training-validation accuracy **gap decreased from 17.05 to 13.25 percentage points**, providing a more controlled starting point for the fine-tuning stage. It is also important to note that the maximum number of epochs was increased to 50 during experimentation. However, model performance stagnated, and no meaningful improvement was observed.

## VII. FINE-TUNING

To improve performance, a fine-tuning phase was executed by unfreezing the last 30 layers of the EfficientNetB0 backbone while keeping early layers locked. This strategy preserves the foundational, low-level shapes (such as edges and corners) learned from ImageNet while allowing these specific layers to adjust their weights to the target poker card features (suits, fonts, and face-card details).

To prevent the loss of pre-trained knowledge, the first model was re-compiled with a lower learning rate of 0.0001, allowing backpropagation to update the newly unfrozen weights more gradually. After 15 epochs, it achieved a final **training accuracy of 86.57%** with a **loss of 0.4311** and a **validation accuracy of 69.61%** with a validation loss of **1.2076**.

After further experimentation with multiple learning rates, including the previous value of 0.0001, the final model was recompiled with a **learning rate of 0.001**, since this value allowed the unfrozen layers to adapt more effectively to the target dataset. After increasing the fine-tuning duration from **15 to 25 epochs**, it achieved a final **training accuracy of 98.06%** with a **loss of 0.2166** and a **validation accuracy of 83.02%** with its lowest validation loss at **0.9571**. This convergence behavior indicates that the fine-tuning strategy successfully specialized the network weights for the poker card classification task while preserving the benefits of the pre-trained EfficientNetB0 feature extractor.

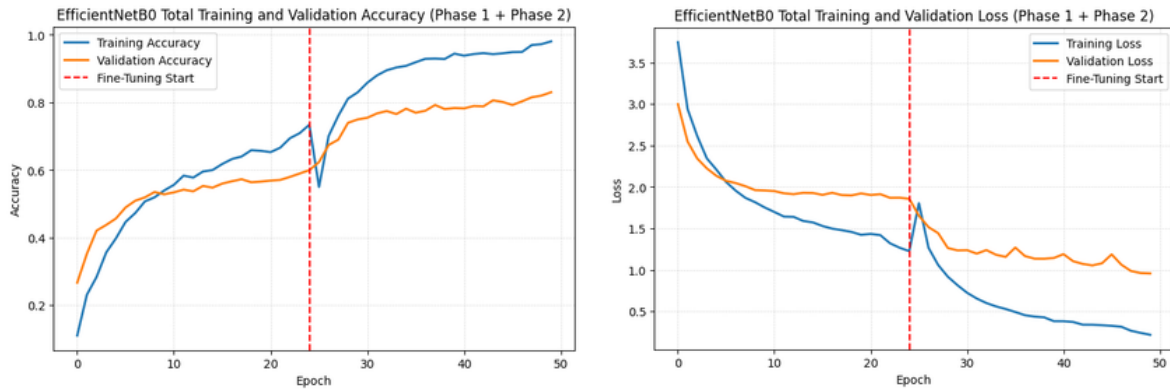


Figure 3. Training and validation accuracy and loss curves.

## VIII. MODEL EVALUATION

The final phase of the project evaluated the trained and fine-tuned model using the independent testing dataset. This provided an unbiased assessment of how the model performs on previously unseen images. The model achieved a **test accuracy of 82.47%** with a **test loss of 0.8783**, indicating that the fine-tuned EfficientNetB0 model was able to correctly classify most unseen poker card images, despite the remaining overfitting observed during training and fine-tuning.

To obtain a more detailed performance analysis, class probabilities were generated for each test image, and the class with the highest probability was selected as the final prediction. These predictions were then compared against the true labels to generate a classification report and confusion matrix.

The classification report showed a macro average **precision of 0.83**, macro average **recall of 0.82**, and macro average **F1-score of 0.82**. The confusion matrix further confirmed that most predictions were concentrated along the main diagonal, meaning that the model correctly classified the majority of card images.

The detailed classification report shows three general performance groups:

- **Top Performers (Aces and Jokers):** These classes achieved some of the strongest results because they have a very clear visual signature. Aces typically include a single prominent central suit symbol and Jokers use a unique illustration. Since they don't look like any other card in the deck, the model recognizes them more consistently.

- **Average Performers (Numerical Cards 2–10):** Most numerical cards fell into a stable performance range, although some individual classes performed especially well, including **four\_spades** with an F1-score of 0.96, **seven\_spades** with an F1-score of 0.96, **nine\_spades** with an F1-score of 0.94, **seven\_clubs** with an F1-score of 0.94, and **seven\_diamonds** with an F1-score of 0.93. However, numerical cards were often confused by the model with one another because they share similar layouts, repeated suit symbols, and geometric patterns. This overlap can make it more difficult for the model to distinguish between ranks or suits in certain cases.
- **Weak Performers (Face Cards: Jacks, Queens, and Kings):** The lowest performance was observed among face cards. The human portraits and royal robes look very similar across all suits, creating complex patterns that make it much harder for the model to isolate the correct suit icon.

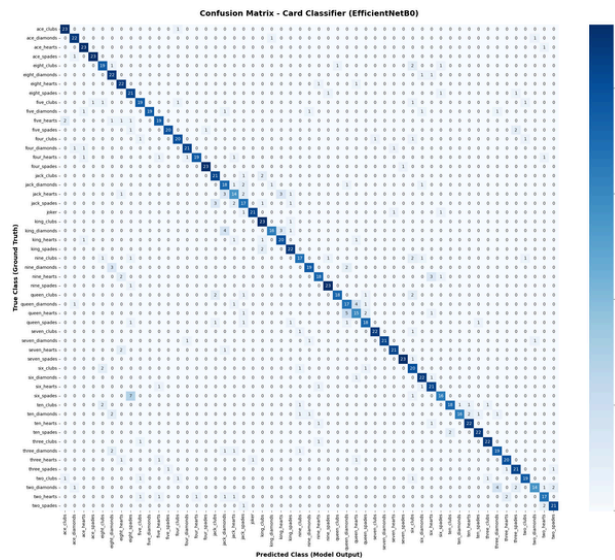


Figure 4. Confusion matrix for the EfficientNetB0 card classifier.

## IX. CONCLUSION

In conclusion, this project successfully developed a deep learning model capable of classifying poker card images across 53 distinct classes using EfficientNetB0 with transfer learning and fine-tuning. The final model reached a test accuracy of 82.47%, showing that the network was able to learn card-specific visual patterns to identify suits and ranks.

However, some limitations remained, including visible overfitting and lower performance on visually complex classes such as Jacks, Queens, and Kings, where similar face-card illustrations made classification more difficult. Future improvements could include expanding the dataset, tuning regularization strategies or researching methods to help the model focus more effectively on suit symbols and fine card details.

Overall, the project demonstrates that transfer learning is an effective approach for image classification tasks with limited training data and resources.

## REFERENCES

Castillo, M., Goeing, B., & Westell, J. (n.d.). Computer vision for card games. Stanford University. <https://cs229.stanford.edu/proj2017/final-reports/5233806.pdf>

Fu, Y. (2019). Keras documentation: Image classification via fine-tuning with EfficientNet. [https://keras.io/examples/vision/image\\_classification\\_efficientnet\\_fine\\_tuning/](https://keras.io/examples/vision/image_classification_efficientnet_fine_tuning/)

Kaggle. (2022). Cards image dataset-classification. <https://www.kaggle.com/datasets/gpiosenska/cards-image-datasetclassification?select=train>

Mittal, K., Gill, K. S., Chauhan, R., Sharma, M., & Sunil, G. (2024). Playing cards classification and detection using sequential CNN model through machine learning techniques using artificial intelligence. In 2024 International Conference on E-mobility, Power Control and Smart Systems (ICEMPS). IEEE. <https://doi.org/10.1109/ICEMPS60684.2024.10559365>

Python Software Foundation. (2025). PEP 8 – Style guide for Python code. Python Enhancement Proposals. <https://peps.python.org/pep-0008/#imports>

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. arXiv. <https://arxiv.org/abs/1905.11946>

TensorFlow. (2024a). `tf.keras.applications.EfficientNetB0`. [https://www.tensorflow.org/api\\_docs/python/tf/keras/applications/EfficientNetB0](https://www.tensorflow.org/api_docs/python/tf/keras/applications/EfficientNetB0)

TensorFlow. (2024b). `tf.keras.callbacks.EarlyStopping`. [https://www.tensorflow.org/api\\_docs/python/tf/keras/callbacks/EarlyStopping](https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/EarlyStopping)

`Tf.keras.layers.GlobalAveragePooling2D` | TensorFlow v2.16.1. (2024c). TensorFlow. [https://www.tensorflow.org/api\\_docs/python/tf/keras/layers/GlobalAveragePooling2D](https://www.tensorflow.org/api_docs/python/tf/keras/layers/GlobalAveragePooling2D)

TensorFlow. (2024d). `tf.keras.callbacks.ReduceLROnPlateau`. [https://www.tensorflow.org/api\\_docs/python/tf/keras/callbacks/ReduceLROnPlateau](https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/ReduceLROnPlateau)

Notebooks del curso:

- `Cats_and_Dogs_Keras.ipynb`
- `Data_augmentation.ipynb`
- `Callbacks for saving models.ipynb`
- `Model Tunning.ipynb`